

1. Implement a user-based collaborative filtering algorithm

```
In [15]: import pandas as pd
import numpy as np
from pathlib import Path
```

```
In [16]: CSV_PATH = "C:\\Users\\Mamata\\Desktop\\PG II\\3rd Semester\\Recommender Systems\\LAB\\Experiment 1\\ratings.csv"
USER_COL_CANDIDATES = ["UserID", "userId", "user_id", "user", "uid"]
ITEM_COL_CANDIDATES = ["MovieID", "movieId", "ItemID", "itemId", "item", "productId", "product_id"]
RATING_COL_CANDIDATES = ["Rating", "rating", "score", "ratings"]
```

```
In [17]: TOP_K_NEIGHBORS = 20      # how many nearest neighbors to use
MIN_OVERLAP = 2                  # min common items to compute Pearson
MIN_SIM = -1.0                   # allow negative sims; set to 0.0 to keep only positive
```

```
In [18]: # 1) LOAD & NORMALIZE COLUMN NAMES / DTYPES
df = pd.read_csv(CSV_PATH)
```

```
In [19]: df
```

```
Out[19]:
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931
...	...	...	...	...
100831	610	166534	4.0	1493848402
100832	610	168248	5.0	1493850091
100833	610	168250	5.0	1494273047
100834	610	168252	5.0	1493846352
100835	610	170875	3.0	1493846415

100836 rows × 4 columns

```
In [20]: def pick_col(cands):
          for c in cands:
              if c in df.columns:
                  return c
          raise KeyError(f"Expected one of {cands} in CSV columns {list(df.columns)}")

u_col = pick_col(USER_COL_CANDIDATES)
i_col = pick_col(ITEM_COL_CANDIDATES)
r_col = pick_col(RATING_COL_CANDIDATES)
```

```
In [21]: # keep only needed cols
df = df[[u_col, i_col, r_col]].copy()
```

In [22]: df

Out[22]:

	userId	movieId	rating
0	1	1	4.0
1	1	3	4.0
2	1	6	4.0
3	1	47	5.0
4	1	50	5.0
...	...	...	...
100831	610	166534	4.0
100832	610	168248	5.0
100833	610	168250	5.0
100834	610	168252	5.0
100835	610	170875	3.0

100836 rows × 3 columns

```
In [24]: # cast IDs to string (avoids KeyError when you pass 'U1' but CSV has 1)
df[u_col] = df[u_col].astype(str)
df[i_col] = df[i_col].astype(str)

# ratings to numeric
df[r_col] = pd.to_numeric(df[r_col], errors="coerce")
df = df.dropna(subset=[r_col])

# (optional) if multiple ratings per (user,item), average them
df = df.groupby([u_col, i_col], as_index=False)[r_col].mean()
```

```
In [26]: print("Data loaded:", df.shape, "rows")
df
```

Data loaded: (100836, 3) rows

```
Out[26]:
```

	userId	movieId	rating
0	1	1	4.0
1	1	1009	3.0
2	1	101	5.0
3	1	1023	5.0
4	1	1024	5.0
...	...	...	...
100831	99	434	3.0
100832	99	435	2.0
100833	99	555	4.0
100834	99	588	5.0
100835	99	592	4.0

100836 rows × 3 columns

```
In [27]: # 2) USER-ITEM MATRIX
R = df.pivot_table(index=u_col, columns=i_col, values=r_col, aggfunc='mean')
# rows: users, cols: items
print("\nUser-Item matrix shape:", R.shape)

# per-user mean (for mean-centering)
user_mean = R.mean(axis=1)
```

User-Item matrix shape: (610, 9724)

```
In [28]: # 3) USER-USER PEARSON SIMILARITY (with min overlap)

# Use pandas corr with min_periods to enforce overlap requirement
# R.T.corr computes correlation between users (columns of R.T == users)
S = R.T.corr(method='pearson', min_periods=MIN_OVERLAP)

# Optional: filter very small similarities (set negatives to 0 if
#you want strictly positive neighbors)
S = S.where(S >= MIN_SIM)

print("\nSample of similarity matrix (Pearson):")
print(S.iloc[:5, :5])
```

```
Sample of similarity matrix (Pearson):
userId      1      10      100      101      102
userId
1      1.000000 -0.037987  0.148877  0.200675 -0.312718
10     -0.037987  1.000000 -0.042619 -0.396059 -0.628619
100     0.148877 -0.042619  1.000000 -0.205196 -0.450377
101     0.200675 -0.396059 -0.205196  1.000000      NaN
102    -0.312718 -0.628619 -0.450377      NaN  1.000000
```



```

In [31]: # 4) PREDICTION & RECOMMENDATION
# -----
def predict_for_user(target_user, top_n=5, top_k=TOP_K_NEIGHBORS):
    target_user = str(target_user)
    if target_user not in R.index:
        raise KeyError(f"User '{target_user}' not found. Available users include e.g.: {list(R.index[:5])}")

    # neighbors sorted by similarity (drop self + NaNs)
    sims = S[target_user].drop(labels=[target_user], errors="ignore").dropna()
    if sims.empty:
        print(f"[WARN] No neighbors with valid Pearson similarity for user {target_user}.")
        return pd.Series(dtype=float)

    sims = sims.sort_values(ascending=False).head(top_k)

    # items not yet rated by target user
    already_rated_mask = R.loc[target_user].notna()
    candidates = R.columns[~already_rated_mask]

    if len(candidates) == 0:
        print(f"[INFO] User {target_user} has rated all items.")
        return pd.Series(dtype=float)

    # mean-centered ratings for neighbors
    R_centered = R.sub(user_mean, axis=0)

    preds = {}
    for item in candidates:
        # neighbors who rated this item
        neigh_ratings = R_centered.loc[sims.index, item].dropna()
        if neigh_ratings.empty:
            continue

        # keep only neighbors we have a similarity for
        common_neigh = neigh_ratings.index.intersection(sims.index)
        w = sims.loc[common_neigh]
        x = neigh_ratings.loc[common_neigh]

        denom = np.abs(w).sum()
        if denom == 0:
            continue

```

```

    # mean-centered prediction then add back target user's mean
    pred = user_mean.loc[target_user] + (w @ x) / denom
    preds[item] = pred

    if not preds:
        print(f"[WARN] No predictable items for user {target_user} (insufficient overlap).")
        return pd.Series(dtype=float)

    recs = pd.Series(preds).sort_values(ascending=False).head(top_n)
    return recs

```

```

In [32]: # 5) EXAMPLE RUN (change the user ID to one that exists in your CSV)
# -----
try:
    user_to_try = R.index[0] if len(R.index) else None
    if user_to_try is None:
        raise RuntimeError("No users found in the dataset after preprocessing.")
    print(f"\nTop recommendations for user '{user_to_try}':")
    print(predict_for_user(user_to_try, top_n=5))
except Exception as e:
    print("\n[ERROR] Step 5 failed:", repr(e))
    print("Troubleshooting checklist:")
    print(" • Ensure the user ID you pass actually exists in the CSV.")
    print(" • Ratings must be numeric; non-numeric values are coerced to NaN and dropped.")
    print(" • Pearson needs overlap: set MIN_OVERLAP lower (e.g., 1) if your data is too sparse.")
    print(" • If you only want positive neighbors, set MIN_SIM = 0.0.")

```

Top recommendations for user '1':

```

3567      6.726379
2599      6.699713
2762      6.699713
30803     6.225754
27611     6.223522
dtype: float64

```

In [33]: # 5) EXAMPLE RUN (change the user ID to one that exists in your CSV)

```
try:
    user_to_try = R.index[1] if len(R.index) else None
    if user_to_try is None:
        raise RuntimeError("No users found in the dataset after preprocessing.")
    print(f"\nTop recommendations for user '{user_to_try}':")
    print(predict_for_user(user_to_try, top_n=5))
except Exception as e:
    print("\n[ERROR] Step 5 failed:", repr(e))
    print("Troubleshooting checklist:")
    print("    • Ensure the user ID you pass actually exists in the CSV.")
    print("    • Ratings must be numeric; non-numeric values are coerced to NaN and dropped.")
    print("    • Pearson needs overlap: set MIN_OVERLAP lower (e.g., 1) if your data is too sparse.")
    print("    • If you only want positive neighbors, set MIN_SIM = 0.0.")
```

Top recommendations for user '10':

3471	5.192857
3543	5.192857
3996	5.192857
4011	5.192857
30803	5.137946

dtype: float64

In [34]: # 5) EXAMPLE RUN (change the user ID to one that exists in your CSV)

```
try:
    user_to_try = R.index[2] if len(R.index) else None
    if user_to_try is None:
        raise RuntimeError("No users found in the dataset after preprocessing.")
    print(f"\nTop recommendations for user '{user_to_try}':")
    print(predict_for_user(user_to_try, top_n=5))
except Exception as e:
    print("\n[ERROR] Step 5 failed:", repr(e))
    print("Troubleshooting checklist:")
    print(" • Ensure the user ID you pass actually exists in the CSV.")
    print(" • Ratings must be numeric; non-numeric values are coerced to NaN and dropped.")
    print(" • Pearson needs overlap: set MIN_OVERLAP lower (e.g., 1) if your data is too sparse.")
    print(" • If you only want positive neighbors, set MIN_SIM = 0.0.")
```

Top recommendations for user '100':

27831	6.105946
3275	6.105946
2542	6.105946
177593	6.010049
174053	6.010049

dtype: float64

In []: